

ACTIVIDAD Nº 2

TÍTULO DE LA ACTIVIDAD: Haciendo persistente la aplicación GuestBook

TEXTO DE LA ACTIVIDAD

En este ejercicio vamos a volver a desplegar nuestra aplicación GuestBook, que realizamos en [Actividad 5.3: Despliegue de la aplicación GuestBook](../modulo5/actividad3.md) y en la [Actividad 6.1: Acceso de la aplicación GuestBook](../modulo6/actividad1.md) para añadirle persistencia a la base de datos redis.

Por lo tanto necesitaremos solicitar un volumen, que se asociará de forma dinámica.

Creando el despliegue de redis para que guarde la información en un directorio

Si estudiamos la documentación de la imagen redis en [Docker Hub](https://hub.docker.com/_/redis), para que la información de la base de datos se guarde en un directorio `/data`` del contenedor hay que ejecutar con docker:

```

```
docker run --name some-redis -d redis redis-server --appendonly yes
```

```

Es decir, hay que crear el contenedor ejecutando el proceso ``redis-server`` con los argumentos ``--appendonly yes``.

Por lo tanto tenemos que cambiar el fichero de definición del Deployment de redis, [``redis-deployment.yaml``](/files/guestbook/plantilla-redis-deployment.yaml) de la siguiente manera:

```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: redis
 labels:
 app: redis
 tier: backend
spec:
 replicas: 1
 selector:
 matchLabels:
 app: redis
 tier: backend
 template:
 metadata:
 labels:
 app: redis
 tier: backend
 spec:
 volumes:
 - name: volumen-redis
 persistentVolumeClaim:
 claimName: xxxxxxxxxxxxxx
 containers:
 - name: contenedor-redis
 image: redis
 command: ["redis-server"]
 args: ["--appendonly", "yes"]
 ports:
 - name: redis-server
 containerPort: 6379
 volumeMounts:
 - mountPath: xxxxxxxxxxxxxx
 name: volumen-redis
```

```

Hemos usado el parámetro `command` para ejecutar el proceso, y el parámetro `args` para indicar los argumentos.

****Nota**:** Los valores `xxxxxxxxxxxx` tendrás que rellenarlos durante la realización de la actividad.

Pasos a seguir

Realiza los siguientes pasos:

1. Crea un fichero yaml para definir un recurso PersistentVolumeClaim que se llame ``pvc-redis`` y para solicitar un volumen de 3Gb.
2. Crea el recurso y comprueba que se ha asociado un volumen de forma dinámica a la solicitud.
3. Modifica el fichero del despliegue de redis, modificando las ``xxxxxxxxxxxx`` por los valores correctos: el nombre del PersistentVolumeClaim y el directorio de montaje en el contenedor (como hemos visto anteriormente es ``/data``).
4. Crea el despliegue de redis. El despliegue de la aplicación ``guestbook`` y la creación de los Services de acceso se hace con los ficheros que ya utilizamos anteriormente: `[`guestbook-deployment.yaml`](files/guestbook/guestbook-deployment.yaml)`, `[`guestbook-srv.yaml`](files/guestbook/guestbook-srv.yaml)` y `[`redis-srv.yaml`](files/guestbook/redis-srv.yaml)`.
5. Accede a la aplicación y escribe algunos mensajes.
6. Comprobemos la persistencia: elimina el despliegue de redis, vuelve a crearlo, vuelve a acceder desde el navegador y comprueba que los mensajes no se han perdido.

Para superar la actividad deberás entregar en un fichero comprimido los siguientes pantallazos:

1. Pantallazo con la definición del recurso PersistentVolumeClaim (**pantallazo1.jpg**).
2. Pantallazo donde se visualicen los recursos ``pv`` y ``pvc`` que se han creado (**pantallazo2.jpg**).
3. Pantallazo donde se vea el fichero yaml modificado para el despliegue de redis (**pantallazo3.jpg**).
4. Pantallazo donde se vea el acceso a la aplicación con los mensajes escritos (**pantallazo4.jpg**).
5. Pantallazo donde se vea que se ha eliminado y se ha vuelto a crear el despliegue de redis y que se sigue sirviendo la aplicación con los mensajes (**pantallazo5.jpg**).